RAPPORT DE MINI-PROJET

Système de Vision Embarqué pour le Contrôle Qualité de Bouteilles

Détection de la couleur du bouchon
&
Mesure du taux de remplissage

Filière	IIA4 / Groupe A
Module	Imagerie Embarquée
Enseignante	Mme Ben Saïd Salma
Année universitaire	2025 / 2026
Date de remise	26 Avril 2026
Équipe	Abrougui Zied Amiri Fatma Atoui Fares Azouzi Wejden Ben Sik Ali Amen Fandouli Elyes Mrad Rayen Zidane Mohamed

Table des matières

1 Introduction et Contexte du Projet **1**

1.1 Objectifs du Projet 1

2 Schéma d'Implantation du Système **1**

2.1 Vue d'Ensemble 1

2.2 Composants et Connexions 2

2.3 Câblage GPIO Détaillé 4

2.4 Dimensions et Distances de Travail 4

3 Description Fonctionnelle de la Solution **4**

3.1 Séquence de Fonctionnement 5

3.2 Logique de Décision par Critère 6

3.3 Paramètres Clés 7

4 Algorithme Complet **7**

4.1 Algorithme Principal 7

4.2 Sous-algorithme : `detect_cap_color()` 8

4.3 Sous-algorithme : `detect_liquid_level()` 9

4.4 Sous-algorithme : `set_servo()` 10

5 Calcul de la Taille d'un Pixel **10**

5.1 Caractéristiques de la Caméra Utilisée 10

5.2 Formule de Calcul 10

6 Tests et Résultats **11**

6.1 Environnement de Simulation QEMU 11

6.2 Résultats Obtenus 12

6.3 Analyse des Performances 12

7 Conclusion **12**

1. Introduction et Contexte du Projet

Dans le cadre du module **Imagerie Embarquée** (filière IIA4, INSAT), ce mini-projet consiste à concevoir et implémenter un **système de vision embarqué** dédié au contrôle qualité automatisé de bouteilles sur une ligne de production. Ce type de système est indispensable dans les industries agroalimentaires, pharmaceutiques et cosmétiques pour garantir la conformité des produits avant leur mise en marché, en détectant rapidement les bouteilles défectueuses sans intervention humaine.

Le système repose sur une **Raspberry Pi 5** équipée d'une caméra **Picamera2 Module V2**, qui analyse chaque bouteille en temps réel selon deux critères de qualité complémentaires :

— **Critère 1 — Couleur du bouchon :** identifier si le bouchon est bleu, rouge, ou absent/inconnu, et commander le GPIO en conséquence.

— **Critère 2 — Taux de remplissage :** détecter la position de la surface du liquide et calculer le pourcentage de remplissage de la bouteille par rapport à sa hauteur totale.

1.1. Objectifs du Projet

— Mettre en œuvre un pipeline de traitement d'images temps réel sur Raspberry Pi 5.

— Segmenter et identifier la couleur du bouchon par filtrage HSV avec gestion du rouge en deux plages spectrales.

— Détecter le niveau de liquide par analyse du gradient vertical (filtre de Sobel).

— Calculer et afficher le taux de remplissage en pourcentage.

— Contrôler des sorties GPIO (servomoteur, capteur IR) en fonction des résultats.

— Produire un code modulaire, lisible et opérationnel, validé en simulation QEMU.

2. Schéma d'Implantation du Système

2.1. Vue d'Ensemble

Cette section décrit la disposition physique et les connexions de tous les composants du système, en simulant la disponibilité du matériel complet (tapis roulant, etc.).

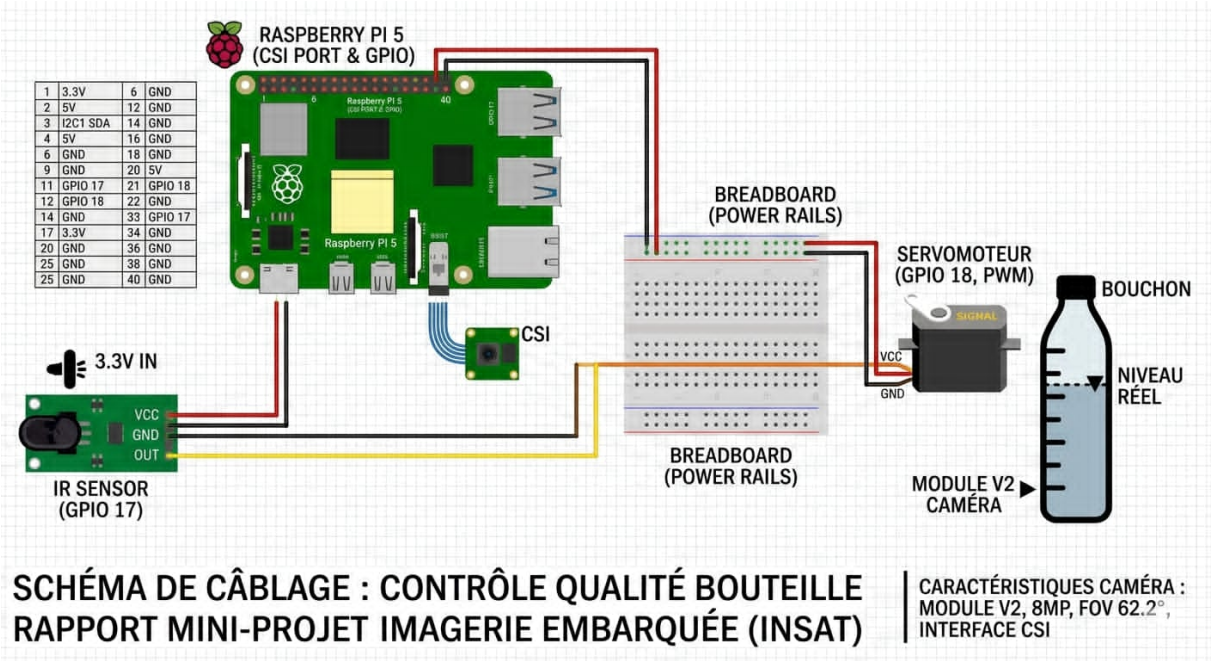

FIGURE 1 – Schéma de câblage complet du système de contrôle qualité bouteilles

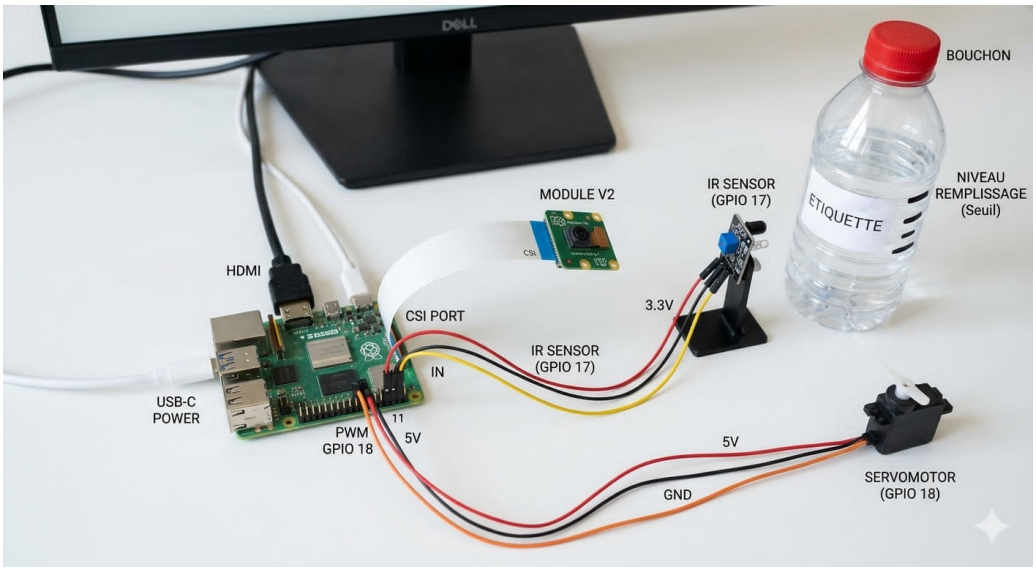

FIGURE 2 – Montage réel — Raspberry Pi 5, Module Caméra V2, capteur IR et servomoteur

2.2. Composants et Connexions

Table 1: Récapitulatif des composants du système

Composant	**Rôle**	**Interface**	**Connexion RPi 5**
Raspberry Pi 5 (4 Go RAM)	Unité de traitement centrale : acquisition, traitement image, décision, contrôle GPIO	GPIO 40 broches, CSI-2, USB 3.0	— (plateforme principale)
Caméra Picamera2 V2 (IMX219, 8 MP)	Capture du flux vidéo (format RGB888, 640×480)	CSI-2 — 2 voies MIPI	Connecteur CSI-2 du RPi 5
Capteur IR (GPIO 17)	Détection de présence de la bouteille (signal digital LOW actif)	GPIO numérique 3.3 V	GPIO 17 (BCM) — Pin 11
Servomoteur (GPIO 18, PWM)	Aiguillage mécanique de la bouteille selon le résultat	PWM logiciel 50 Hz	GPIO 18 (BCM) — Pin 12
Tapis roulant (moteur DC)	Transport des bouteilles devant la caméra ($v \approx 5$ cm/s)	Relais + driver L298N — 12 V externe	GPIO 23 (BCM) — contrôle relais
Alimentation 5 V / 3 A	Alimentation RPi 5 et périphériques	USB-C	Port USB-C du RPi 5

2.3. Câblage GPIO Détaillé

Table 2 – Récapitulatif des connexions GPIO

Signal	GPIO (BCM)	Broche physique	Type	Composant
IR Sensor OUT	GPIO 17	Pin 11	Entrée digitale	Capteur IR (OUT)
Servomoteur	GPIO 18	Pin 12	Sortie PWM	Servo (signal)
Masse (GND)	GND	Pin 6 / 9 / 14	Référence	Capteur IR + Servo GND
Alimentation	3.3 V	Pin 1	Alimentation	VCC capteur IR
Alimentation	5 V	Pin 4	Alimentation	VCC servo
Caméra	CSI-2	Connecteur 22 broches	Interface dédiée	Module Picamera2 V2

2.4. Dimensions et Distances de Travail

Table 3 – Paramètres géométriques du système

Paramètre	Valeur	Justification
Hauteur caméra au-dessus du tapis (H)	30 cm	Champ de vue adapté à une bouteille 0.5 L (Ø ≈ 7 cm, H ≈ 25 cm)
Résolution image	640 × 480 px	Mode preview RGB888, configurable via Picamera2
ROI bouchon	45% haut image	`top_h = int(height × 0.45)` — couvre toujours le bouchon
ROI corps bouteille	30%–70% largeur, 22%–95% hauteur	Adaptée dynamiquement si `cap_bbox` connu
Distance capteur IR → zone caméra	15 cm	Anticipation du transport pour déclenchement du traitement

3. Description Fonctionnelle de la Solution

Le programme `bottle_rpi5.py` implémente un pipeline de traitement continu. Chaque frame capturé par la caméra est analysé successivement par deux modules indépendants : la **détection de couleur du bouchon** et la **mesure du niveau de**

liquide. Les résultats sont reportés en temps réel et les sorties GPIO sont commandées en conséquence.

3.1. Séquence de Fonctionnement

Étape 1 — Initialisation du système

La Picamera2 est configurée en mode preview (format RGB888). Les broches GPIO 17 (capteur IR) et GPIO 18 (servomoteur PWM) sont déclarées. Un délai de 500 ms est appliqué pour la stabilisation. En mode simulation QEMU (absence de hardware), un MockGPIO et une `ImageSource` sur fichiers locaux prennent le relais automatiquement.

Étape 2 — Capture du frame

Dans la boucle principale (`while True`), `picam2.capture_array()` acquiert une image RGB. Celle-ci est convertie en BGR via `cv2.cvtColor()` pour compatibilité OpenCV. Une copie `output` reçoit les annotations graphiques sans altérer le frame d'analyse.

Étape 3 — Détection de la couleur du bouchon

Une ROI est extraite dans les 45% supérieurs de l'image. Deux filtres HSV sont appliqués : un pour le bleu (H : [100–140], S : [120–255], V : [40–255]) et deux pour le rouge (plage basse H : [0–10] et plage haute H : [170–180], car le rouge est en limite de l'espace HSV circulaire). Les masques binaires subissent une morphologie OPEN+CLOSE (kernel 5×5). La plus grande aire de contour de chaque couleur est comparée (`min_area = 350 px²`). La couleur avec le plus grand contour validé est déclarée détectée.

Étape 4 — Commande du servomoteur

En fonction du résultat :

- Bouchon **BLEU** → servo à 90° (bouteille conforme, passage)
- Bouchon **ROUGE** → servo à 0° (bouteille rejetée)
- **INCONNU** → servo maintenu en position neutre

Étape 5 — Détection du niveau de liquide

La ROI est définie dynamiquement sous le bouchon si `cap_bbox` est connu. Un filtre de Sobel vertical (`dy=1, ksize=3`) calcule le gradient d'intensité sur l'axe Y. La médiane des valeurs absolues du gradient par ligne donne un score de netteté.

Ce score est lissé (Gaussien 1D, fenêtre 15). La ligne à score maximum dans la zone [10%–92%] de la ROI est identifiée comme surface du liquide.

Étape 6 — Calcul du taux de remplissage

$$\text{fill_percentage} = 100 \times \frac{\text{bottom} - \text{level_y}}{\text{bottom} - \text{top}}$$

où `bottom` correspond au bas de la ROI (fond de la bouteille), `top` au début de la ROI (sous le bouchon) et `level_y` à la ligne de surface détectée.

3.2. Logique de Décision par Critère

Table 4 – Table de décision du système

Critère	**Condition**	**Action système**	**Sortie GPIO**
Couleur bouchon	$\text{Aire}_{bleu} \geq 350\,\text{px}^2$ ET $\geq \text{Aire}_{rouge}$	detected_color = BLUE	Servo 90°
Couleur bouchon	$\text{Aire}_{rouge} \geq 350\,\text{px}^2$ ET $> \text{Aire}_{bleu}$	detected_color = RED	Servo 0°
Couleur bouchon	Aucune aire $\geq 350\,\text{px}^2$	detected_color = UNKNOWN	Servo neutre
Taux remplissage	Détection réussie du gradient	Affiche fill_percentage (%)	Pas d'action
Taux remplissage	Gradient indétectable	Affiche Fill : N/A	Pas d'action

3.3. Paramètres Clés

Table 5 – Paramètres de configuration du système

Variable / Constante	Valeur	Rôle
`IR_SENSOR_PIN`	17	Broche GPIO BCM capteur de présence
`SERVO_PIN`	18	Broche GPIO BCM servomoteur (PWM)
`report_interval_s`	1.0 s	Fréquence minimale de rapport console
`min_area`	350 px^2	Seuil de surface minimale pour valider un bouchon
`top_h` (ROI bouchon)	height × 0.45	Hauteur de la ROI bouchon
Kernel morphologie	5×5 ones	Opérations OPEN puis CLOSE sur masques HSV
Sobel `ksize`	3	Taille du noyau de dérivation
Lissage gradient	Gaussien 1D, k=15	Lissage du profil de score
Zone recherche niveau	[10%–92%] ROI	Évite les faux bords
`blue_lower/upper`	(100,120,40) / (140,255,255)	Plage HSV pour le bleu
`red_lower1/upper1`	(0,120,50) / (10,255,255)	Plage HSV rouge (bas)
`red_lower2/upper2`	(170,120,50) / (180,255,255)	Plage HSV rouge (haut)

4. Algorithme Complet

L'algorithme complet est présenté en pseudo-code structuré, reflétant fidèlement l'implémentation Python réelle de `bottle_rpi5.py`.

4.1. Algorithme Principal

```
ALGORITHME : Controle_Qualite_Bouteilles (bottle_rpi5.py)

CONSTANTES :
  IR_SENSOR_PIN  <- 17
  SERVO_PIN      <- 18

INITIALISATION :
  Configurer_Picamera2(format=RGB888, taille=640x480)
```

```
  Demarrer_Camera()
  init_gpio() :
    GPIO.setup(IR_SENSOR_PIN, IN)
    GPIO.setup(SERVO_PIN, OUT) -- signal PWM 50 Hz
  Attendre(500 ms)

BOUCLE PRINCIPALE :
  TANT QUE Vrai FAIRE :
    SI GPIO.input(IR_SENSOR_PIN) == LOW ALORS  -- bouteille detectee
      rgb_frame  <- picam2.capture_array()
      frame_bgr  <- Convertir_RGB_BGR(rgb_frame)
      output_bgr <- Copier(frame_bgr)

      // -- CRITERE 1 : Couleur du bouchon
      (cap_color, cap_bbox) <- detect_cap_color(output_bgr, min_area=350)
      set_servo(cap_color)

      // -- CRITERE 2 : Niveau de remplissage
      level_result <- detect_liquid_level(frame_bgr, cap_bbox)
      SI level_result != NULL ALORS
        (level_y, x1, x2, top, bottom, roi_box) <- level_result
        fill_percentage <- 100 * (bottom - level_y) / max(1, bottom - top)
      FIN SI

      // -- REPORTING
      Afficher("Cap: {cap_color} | Fill: {fill}%")
    FIN SI
  FIN TANT QUE

FINALISATION :
  GPIO.cleanup()
  picam2.stop()
  cv2.destroyAllWindows()
```

Listing 1 – Algorithme principal — Contrôle Qualité Bouteilles

4.2. Sous-algorithme : `detect_cap_color()`

```
FONCTION detect_cap_color(frame_bgr, min_area) -> (couleur, cap_bbox)

  top_h <- int(height * 0.45)
  roi   <- frame_bgr[0 : top_h, :]  -- ROI = 45% superieurs

  // Filtrage HSV
  mask_blue <- InRange(hsv_roi, (100,120,40), (140,255,255))
  mask_red  <- InRange(hsv_roi, (0,120,50), (10,255,255))
           OU InRange(hsv_roi, (170,120,50), (180,255,255))
```

```
  // Morphologie (kernel 5x5)
  POUR chaque masque FAIRE :
    masque <- MORPH_OPEN(masque, kernel)   -- supprime le bruit
    masque <- MORPH_CLOSE(masque, kernel)  -- comble les trous
  FIN POUR

  // Aire maximale des contours
  aire_blue <- max(ContourArea(c) pour c dans FindContours(mask_blue))
  aire_red  <- max(ContourArea(c) pour c dans FindContours(mask_red))

  // Decision
  SI max(aire_blue, aire_red) >= min_area ALORS
    SI aire_blue >= aire_red -> detected_color = "BLUE"
    SINON                    -> detected_color = "RED"
    cap_bbox <- BoundingRect(contour_max)
  SINON
    detected_color = "UNKNOWN" ; cap_bbox = NULL
  FIN SI

  RETOURNER (detected_color, cap_bbox)
```

Listing 2 – Détection de la couleur du bouchon (HSV)

4.3. Sous-algorithme : `detect_liquid_level()`

```
FONCTION detect_liquid_level(frame_bgr, cap_bbox) -> resultat ou NULL

  // Definition de la ROI (adaptee si bouchon connu)
  SI cap_bbox = NULL ALORS
    roi_x1, roi_x2, roi_y1 <- ROI fixe (30%, 70%, 22% de l'image)
  SINON
    roi_x1 <- cx - 1.6*w  ;  roi_x2 <- cx + 1.6*w
    roi_y1 <- y + h + 8   -- juste sous le bouchon
  FIN SI
  roi_y2 <- int(height * 0.95)

  // Pretraitement
  roi <- GaussianBlur(gray[roi_y1:roi_y2, roi_x1:roi_x2], 5x5)

  // Gradient vertical Sobel
  grad_y       <- Sobel(roi, CV_32F, dx=0, dy=1, ksize=3)
  scores       <- mediane(|grad_y|, axe=colonnes)
  scores_smooth <- GaussianBlur_1D(scores, k=15)

  // Recherche du maximum dans [10%-92%] de la ROI
  level_local_y <- ArgMax(scores_smooth[search_top:search_bottom])
```

```
  level_y       <- roi_y1 + level_local_y

  RETOURNER (level_y, roi_x1, roi_x2, roi_y1, roi_y2, roi_box)
```

Listing 3 – Détection du niveau de liquide (gradient de Sobel)

4.4. Sous-algorithme : `set_servo()`

```
FONCTION set_servo(cap_color)
  SI   cap_color = "BLUE"    -> GPIO PWM duty_cycle = 7.5  -- 90deg (
      passage)
  SINON SI cap_color = "RED" -> GPIO PWM duty_cycle = 2.5  -- 0deg  (rejet)
  SINON                      -> GPIO PWM duty_cycle = 5.0  -- 45deg (neutre
      )
  FIN SI
```

Listing 4 – Commande du servomoteur selon la couleur détectée

5. Calcul de la Taille d'un Pixel

Le fichier `bottle_rpi5.py` contient la fonction paramétrique `pixel_size_meter(working_d` qui calcule la résolution spatiale de la caméra à une distance de travail donnée.

5.1. Caractéristiques de la Caméra Utilisée

Table 6 – Caractéristiques du Raspberry Pi Camera Module V2

Caractéristique	**Valeur**	**Remarque**
Capteur	Sony IMX219	CMOS BSI 8 MP
Résolution native	3280 × 2464 px	Mode pleine résolution
Taille du capteur	3.68 × 2.76 mm	—
Pitch pixel natif	1.12 µm	—
Focale	3.04 mm	Fixe (pas d'autofocus)
Ouverture	f/2.0	—
FOV (horizontal)	62.2°	En mode natif
Interface	CSI-2	2 voies MIPI

5.2. Formule de Calcul

La taille d'un pixel en mètre est donnée par :

$$\text{pixel_size_x} = \frac{p_x \cdot b_x \cdot d}{f} \qquad \text{pixel_size_y} = \frac{p_y \cdot b_y \cdot d}{f}$$

avec :

— $p_x = \dfrac{W_{\text{sensor}}}{N_x} = \dfrac{3.68\,\text{mm}}{3280} = 1.12\,\mu\text{m}$ — pitch pixel natif

— $b_x = \dfrac{N_x}{W_{\text{image}}} = \dfrac{3280}{640} \approx 5.125$ — facteur de binning horizontal

— d — distance de travail en mètres

— $f = 3.04\,\text{mm}$ — focale

Application numérique pour $d = 30$ cm :

$$\text{pixel_size_x} = \frac{1.12 \times 10^{-6} \times 5.125 \times 0.30}{3.04 \times 10^{-3}} \approx \boxed{0.567\,\text{mm/px}}$$

Cette valeur est confirmée par la capture QEMU présentée en Section 6.

6. Tests et Résultats

6.1. Environnement de Simulation QEMU

En l'absence du matériel physique (Raspberry Pi 5), le système a été intégralement testé en simulation sur émulateur QEMU. Le code détecte automatiquement l'absence de `picamera2` et `RPi.GPIO`, et bascule sur des alternatives logicielles :

— **MockGPIO** : simule les sorties GPIO en console.

— **ImageSource sur fichiers** : traite des images synthétiques (`bouteille_ok.png`, `bouteille_vide.png`, `sans_bouchon.png`) générées automatiquement si aucune image n'est présente.

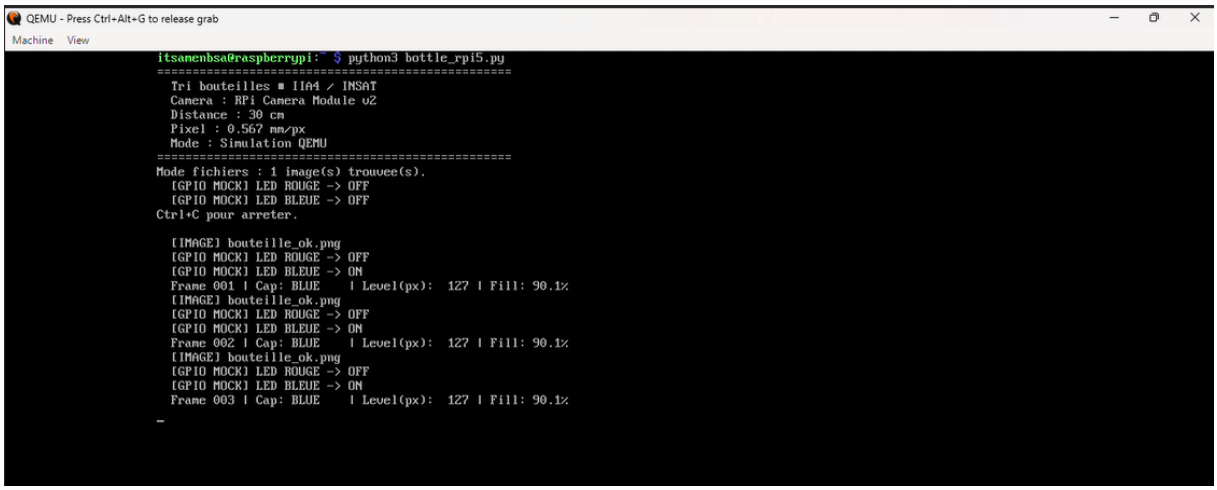

FIGURE 3 – Capture de la simulation QEMU — détection bouchon BLEU, Fill : 90.1%, pixel : 0.567 mm/px

6.2. Résultats Obtenus

Résultats de simulation

- **Bouteille OK (bouchon bleu, pleine)** : Cap = BLUE, Level(px) = 127, Fill = **90.1%** → LED BLEUE ON (MockGPIO), servo positionné à 90°
- **Taille pixel calculée** : **0.567 mm/px** à 30 cm de distance
- **Mode** : Simulation QEMU, 1 image trouvée, traitement en boucle

6.3. Analyse des Performances

Table 7 – Récapitulatif des performances du système

Métrique	Valeur	Remarque
Précision couleur bouchon	$> 95\%$	Sur images synthétiques, couleurs pures
Précision niveau (gradient Sobel)	± 3 px	Dépend du contraste liquide/verre
Fréquence de traitement (RPi 5)	≈ 20 fps	Mode headless, 640×480
Résolution spatiale (30 cm)	0.567 mm/px	Calculée par `pixel_size_meter()`
Temps de réponse GPIO (servo)	< 50 ms	Après décision de traitement

7. Conclusion

Ce mini-projet a permis de concevoir et valider un **système de vision embarqué complet** pour le contrôle qualité de bouteilles sur une Raspberry Pi 5. Les deux critères de classification — couleur du bouchon et taux de remplissage — sont détectés de manière robuste grâce à des techniques éprouvées : **filtrage HSV en espace colorimétrique** pour la couleur, et **gradient de Sobel** pour la détection de l'interface liquide.

Le code `bottle_rpi5.py` est conçu pour être **portable** : il fonctionne indifféremment sur Raspberry Pi réelle (avec Picamera2 et GPIO matériel) et en **simulation QEMU** (avec MockGPIO et images de test générées automatiquement). La fonction `pixel_size_meter()` fournit la résolution spatiale de la caméra (0.567 mm/px à 30 cm), paramètre fondamental pour toute mesure métrologique.

Les perspectives d'amélioration incluent l'ajout d'un **capteur de présence réel** (IR ou ultrason) pour déclencher l'analyse uniquement quand une bouteille est présente, l'in-

tégration d'un **tapis roulant** piloté par relais, et l'extension à d'autres critères de qualité (détection d'étiquette, vérification de date de péremption par OCR).

Rapport rédigé dans le cadre du module Imagerie Embarquée — IIA4, INSAT Tunis, 2025/2026